

API and Project Files Reference

FitBuddy AI Fitness Plan Generator

The documented endpoints, data layout and safe configuration summary.

HTTP route quick reference

Method and route	Purpose
GET /	Render the profile form.
POST /generate-workout	Generate and save a plan and tip; render results.
POST /submit-feedback	Revise the latest plan for a user ID.
GET /api/health	Return service status and active AI mode.
POST /api/plans	Create and store a plan through JSON.
POST /api/plans/{user_id}/feedback	Revise the latest saved plan.
GET /api/plans/{user_id}	Return plan history and feedback.
GET /api/admin/users	Token-protected coach data.
DELETE /api/admin/users/{user_id}	Delete a user and associated plans.

SQLite data map

Table	Stored fields
users	id, user_id, username, age, weight_kg, goal, intensity, created_at.
workout_plans	id, user_id, original_plan, updated_plan, nutrition_tip, feedback, created_at, updated_at.

Dependencies

requirements.txt lists FastAPI, Uvicorn, SQLAlchemy, Jinja2, python-multipart, python-dotenv, pydantic-settings and google-genai with bounded major versions.

Configuration keys

The project settings include GEMINI_API_KEY, GEMINI_WORKOUT_MODEL, GEMINI_TIP_MODEL, DATABASE_URL, APP_NAME, ADMIN_TOKEN and ALLOW_DEMO_AI. Store private values only in .env. Use placeholders in shared examples.

Project file safety

The supplied .gitignore excludes .venv/, Python bytecode, .env, *.db, pytest cache and VS Code logs. The supplied .env.example has non-empty configuration values, including a Gemini key according to the project guide; replace secret values with placeholders and rotate the exposed key before sharing. The supplied fitbuddy.db contains stored profile and plan records, so this package documents its schema but does not copy its records.